Opto tips guide

Some little known features and approaches explained:

- Commands

- Pure initialization jobs

- Command line arguments

- Function caching

- Troubleshooting

- Searching multiple parameter regions and choosing the best signal among them

- Shimming tip

- Running jobs from the command line

- hover texts showing curr pass values

- describe the order of init panel and setup and teardown etc including when things happen

- using the plotting script

- better CP approaches possible (Bruker only?)

- using the -R, -L, and -M options in pass parameters limits definitions

- a tip about using OPTO with TopSpin

Commands

During the run of Opto jobs you can insert commands into the process at defined points. You can run commands at job **Set Up** before any job passes have run. You can run commanda at job **Tear Down** after all the job passes have run. Or you can tag a specific pass with commands to run **Before** or **After**. You can also use **Execute Command** at any time a job is not running.

There are five kinds commands: **console()**, **spec()**, **win()**, **linux()**, and **opto()** commands.

console() commands:

A console() command is designed to output a string to the terminal window. Ant text between the parentheses are printed raw to the terminal. This kind f command can be useful for debugging your command logic during a job's run time.

spec() commands:

A spec() command is a command sent to the spectrometer. Any text between the parentheses are sent as is directly to the spectrometer software. A OVJ example might be something like spec(nt=16) which sets the number of transients (or scans) to 16.

linux() commands: calling a linux() command with text between the parentheses will try to run the text as a linux command on the current os. For example: linux(ls -al /)

win() commands: calling a win() command with text between the parentheses will try to run the text as a win command on the current os. For example: win(dir \)

Note that linux() and win() commands have an optional argument. "TO" - the amount of time to allow the windows or linux command to complete before timing out with an error. This "TO" argument when not present defaults to 30 seconds.

Examples:

run a basic linux command:

linux(ls -al / > /tmp/dirListing.txt)

The command is allowed 30 seconds to successfully complete.

run a linux command with a timeout argument:

linux(TO 240 slowCommand.sh)

The command is allowed 240 seconds to successfully complete.

opto() commands are special internal commands you can call to manipulate internal Opto state. They are delineated below. One word: in opto() commands all parameters are separated by spaces (not commas).

opto(zero)
Sets all the pulse sequence parameters or shims to 0. Usually you do this in a Set Up Command or the very 1st Before Command of a shim job.

opto(saveP name)

You can snapshot all the current shims or params and save them with a name for later retrieval (with loadP) or later reference (like in an 'if' command).

opto(saveP region1)

Note that during every pass, at various key points, the system does automatic saveP calls. The two save names of greatest importance are p<n>-post-opt and p<n>-end. For pass 1 this looks like p1-post-opt and p1-end. The -post-opt snapshot contains all the parameters and the best solution signal (named as “eval”) right after pass optimization has run. The -end snapshot contains all the parameters as recorded when the pass totally ended. Because of things like “After” commands these two snapshots can diff. Note that the “eval” value is not in the -end snapshot because the “eval” value is only associated with the actually optimized parameter values as stored in the -post-opt snapshot.

You can refer to saved variables (for example in an 'if' command) as p7-start.aX or p505-post-opt.eval or myRegion.bX etc. You can use the name or the nickname of a parameter so in the TopSpin version of Opto p1.CNST7 and p1.whatever_I_nicknamed_my_cnst7 should both work.

opto(loadP passName)

loadP will load the current pulse sequence params or shims from a previously stored saveP.

Examples: opto(loadP p4-end) or opto(loadP savedRegion1) etc

opto(loadP passName varName1 varName2 varName3 etc)

This version of loadP will only load varName1, varName2, and varName3 etc shims/params from a previous saveP.

opto(loadPMin passName1 passName2 passName3 etc)

Load the values of the parameter set with the smallest evaluation (signal) value from a given set of pass names.

Examples:

opto(loadPMin p1 p2 p7 p1008)

opto(loadPMin region1 region2 region3)

This version is named a Min because it checks for the smallest evaluation (signal) among all the passes. For instance, if the passes were minimization passes.

opto(loadPMax passName1 passName2 passName3 etc)

Load the values of the parameter set with the largest evaluation (signal) value from a given set of pass names.

Examples:

opto(loadPMax p1 p7)

opto(loadPMax region1 region23)

This version is named a Max because it checks for the largest evaluation (signal) among all the passes. For instance, if the passes were maximization passes.

opto(if (testClause) command1() command2())

In the if command **command1** is the command to run if the **testClause** evaluates to true.

Note that **command2** is optional and is the command to run if the **testClause** evaluates to false.

The **testClause** is always in parentheses. You can test things like:

(p1-post-opt.eval >= 100123.0)

(p3.aX >= p2.aX and p3.aX <= p1.aX)

(p2.aX < (p1.aX * p1.aX))

(3.2 < p1.bX and p1.bX < log(10000))

testClauses can get arbitrarily complex. You can use connectors such as **and or not** with operators **+ - * / ^** and functions **abs exp log sqrt cbrt sin cos tan asin acos atan** and constants **E GAMMA PHI PI MAS.** (**MAS** is the magic spinning angle constant in radians). Note you can nest things with parentheses to make sure your order of operations is respected.

The true and false branches of the if command are themselves commands. They can be:

- another opto() command
- a spec() command
- a linux() command
- a win() command

So a properly formatted 'if' command might look like this:

opto(if (p1.aX < p2.aX) spec(set some var or condition) linux(echo 'Oh no!" > /tmp/messages-from-opto))

Note that things can get interesting in that you can nest if commands inside other if commands. The syntax gets hard to read / follow but it works.

Think of structuring something like this in one nested 'if' statement:

```
 if A
  then
   if B
    then A and B
    else A and not B
  else
   if B
    then not A and B
    else not A and not B
```

opto(if (p1.A > 0) opto(if (p1.B > 0) linux(echo A and B > /tmp/info) linux(echo A and not B > /tmp/info)) opto(if (p1.B > 0) linux(echo not A and B > /tmp/info) linux(echo not A and not B > /tmp/info)))

Pure initialization jobs

When you run an opto job you can export the resulting parameters to what is called a pure initialization job. For example, with a job that has parameters X, Y, and Z you can specify on the Control Panel of the job to Export (let's say) X and Y.

Let's say the job is called MyJob.opto-job. When you run the job the final values of X and Y are stored in a new job in the same location called MyJob-last-run.opto-job.

Running such a job has the effect of loading the spectrometer with the values for X and Y as stored from the last run of the original job. This job does nothing but pure parameter initialization. The initialization can be useful to propagate variables from multiple optimization runs into more complicated optimization runs in the downstream process.

Command line arguments

Get the command line help screen by doing this:

```
opto.sh --help
```

Opto 1.2.28 Help

Get info about the Opto version:

```
opto.sh --version
```

Start opto: (launches gui)

```
opto.sh
```

Start opto and load job: (launches gui)

```
opto.sh the-job.opto-job
```

Start opto, load job, and run it in the console terminal:

```
opto.sh --run the-job.opto-job
```

Other command line flags of interest:

--cache and --no-cache

Opto can cache values from the spectrometer and reuse them to speed computation. This is called function caching.

--no-init

Opto can skip the Init panel initialization of the job specified on the command line.

--debug

Do not talk to the spectrometer hardware. This is mostly a developer testing option.

What is function caching?

As the above paragraph about the –cache and –no-cache command line arguments show OPTO does what it calls function caching. The default behavior is for caching to be enabled.

Every time the numerical solvers in OPTO need to measure the experiment with a set of values the result can be cached. The next time this point is visited the

previous result is just returned without interacting with the spectrometer. This has the benefit of speeding up the optimization process but at the cost of sometimes investigating noisy points too much.

To avoid forever being stuck with a noisy point the caching sometimes (about 15% of the time) will ask the spectrometer to once again measure the value at the current point from the spectrometer. Noisy peaks can then be corrected. However you may want to turn off caching so that noisy points are always corrected so they have less effect upon the optimization process.

Turning on and off caching is done at the command line using –-cache and –no-cache.

Troubleshooting

By default Opto communicates on port 30811. If this conflicts with other software on your system you can set the OPTO_PORT environmental variable to another free port number.

Searching multiple parameter regions and choosing the best signal

Imagine you have a job with two parameters (X and Y). You want to find the best Y over the X range -10 to 10. But you also know there is a confounding point at 0 of the X range. You can find the best (X,Y) pair by optimizing two different regions and choose the best afterwards. An example might illustrate this best.

You can setup your job with two passes.

As a **Before Command** to pass 1 you want to save the baseline parameters. You will return to this baseline so it is important to record it. You might do something like **opto(saveP baseline)** in a **Before Command** to pass 1.

Then in Pass 1 you might tell Opto to optimize X and Y but to restrict the Min and Max of X to -10 and -0.1. At the end of the pass, after optimization of the pass but before **After Commands** have run, Opto automatically does an **opto(saveP p1)** command.

Then in Pass 2 you might tell Opto to optimize X and Y but to restrict the Min and Max of X to 0.1 and 10. At the beginning of pass 2, in a **Before Command**, you would do **opto(loadP baseline)** to restore the same initial conditions as pass 1. At the end of the pass, after optimization of the pass but before **After Commands** have run, Opto automatically does an **opto(saveP p2)** command.

At the end of Pass 2 in an **After Command** you can do **opto(loadPMax p1 p2)**. LoadPMax will load the parameters from the list of given parameter set names with the highest signal. After this has run your job will have the best set of parameters loaded; the region that produced the highest signal.

Shimming tips

At the beginning of a shim job you should consider running **opto(zero)** as a Set Up command for the job. This will zero all the shims before any optimization is done and will give you a neutral starting point.

Running jobs from the command line

Opto jobs can be run from the command line without a GUI. You might want to do this because you can build up a library of composable jobs that you can run in concert to solve complicated optimization problems. You can write scripts (such as in bash or python) that can call multiple jobs and record your workflow for later use.

For example, I might have a set of a 1-d, a 2-d, and a 3-d jobs. The 1-d job finds the best (u). The 2-d job finds the best (u,v) pair starting at the existing (u). And the 3-d job finds the best (u,v,w) triple starting at the existing (u,v).

A bash script that can emulate this workflow:

```
opto.sh --run 1-d.opto-job
if [ $? -eq 0 ]; then
   opto.sh --run 2-d.opto-job
   if [ $? -eq 0 ]; then
      opto.sh --run 3-d.opto-job
      if [ $? -eq 0 ]; then
         echo "All commands succeeded."
      else
         echo "3-d job failed to complete: exit status: $?"
      fi
   else
      echo "2-d job failed to complete: exit status: $?"
   fi
else
   echo "1-d job failed to complete: exit status: $?"
fi
```

One of the benefits of a workflow approach is the ability to let Opto run for a long time unattended.

As part of running jobs you can create pure initialization jobs via checking Export options (see above). They end up with names like <original-job>-last-run.opto-job. For instance, in our example above, the 2-d job run could create a 2-d-last-run.opto-job.

Once you've gathered these last-run job files you can skip the running of the original job altogether and simply run a <original-job>-last-run.opto-job to push the last known best values to the spectrometer.

----

document pure spectrometer manipulation jobs (like changing the experiment)

Changing experiments during a job run, via a Before Command or After Command of a running job can confuse Opto on where to place the log file. By default it opens a log file at the beginning of the job in the current experiment directory. If a Before or After command changes the current experiment then any output files will go into the correct place **except for the already open log file!**

If this is bothersome note that you can remove the Before or After command from the job, write a script that will run 3 jobs from the command line. Job 1 can be the original job before the experiment changing before/after command happens. Job 2 can be made as a job that does nothing except change the running experiment. Job 3 can be all the tasks remaining from the original job after the Before/After command ran that would change the experiment. Then in the script you might run job 1, run job 2 (experiment switcher job), and then run job 3. Log files should now go in the right place for job 3.

Note that you can also design job 2 to do any additional small experimental checks you might want to do between the job 1 and job 3. Or run set up / tear down / before / after commands that might call external bookkeeping applications etc.

Hover texts showing current pass values

Note that when a job is running you can hover the mouse over the name of a variable in the control panel or the pass panel and its current value is displayed as hover text. To limit the number of calls the spectrometer has to handle, and to speed the optimization process, these values are only updated at the end of each pass. But it gives you another way to see how parameters are changing. Other

ways include watching output in the plotting scripts and also watching the values being displayed in the terminal window while the job is running.

Describe the order of the initialization from the Control panel and the set up commands and the tear down commands etc including when parameters are saved

When a job runs things progress in this order:

opto(saveP job-start)

- Job Set Up commands are run
  and then opto(saveP job-post-set-up)

- Job Control Panel initialization is run (but can be skipped with a command line flag)
  and then opto(saveP job-post-init)

- then each pass is run

  opto(saveP p<n>-start)

  - Pass Before commands are run
    and then opto(saveP p<n>-post-before)

  - The optimization for the pass is run
    and then opto(saveP p<n>-post-opt)
    (NOTE: this is where the pass optimization “eval” variable is stored)

  - Pass After commands are run
    and then opto(saveP p<n>-post-after)

  opto(saveP p<n>-end)
    which can also be referred to as just p<n>

- Job Tear Down commands are run
  and then opto(saveP job-post-tear-down)

And finally:
  opto(saveP job-end)

Better CP approaches possible (Bruker only?)

Using the -R, -L, and -M options in pass parameters limits definitions

When you are in setting up the per pass limits in a numbered pass panel you can use a few shortcuts to help you.

If you set the min and max range of a parameter you can specify the start point for that parameter in the Value field. But you can also use the **L**, **R**, and **M** shortcuts (or **left**, **right**, and **midpt** if you want to use more complete names). If you specify **L** (or **left**) the parameter will start at the left edge of the min/max interval. If you specify **R** (or **right**) the parameter will start at the right edge of the min/max interval. If you specify **M** (or **midpt**) the parameter will start at the midpoint of the min/max interval.

A tip about using OPTO with TopSpin

If you need to process data or view the ongoing experiment in real time, you can start a new TopSpin session by typing "newtop" in the TopSpin command-line window. This will open an independent session that will not interfere with the OPTO run. You can also move this new GUI window to a different desktop if the active cursor shifts back to the main TopSpin window.